



# Inventra - Intelligent Inventory Management System

Overview of Module 1: Authentication & Module 2: Product & Inventory Management

# Module 1 - Authentication Overview

- ▶ **Explanation:**
- ▶ Authentication is used to verify **who the user is**
- ▶ Only registered users can access the system
- ▶ Prevents unauthorized access
- ▶ It is the first security layer of the application

# Classes Used in Authentication Module

- ▶ **Explanation:**
- ▶ The following classes work together to implement authentication:
- ▶ AuthController - handles user requests
- ▶ AuthService - contains authentication logic
- ▶ User - user entity
- ▶ UserRepository - database access
- ▶ JWTUtility - token generation
- ▶ EmailService - password reset email

# Signup - Purpose

- ▶ **Explanation:**
- ▶ Admin registers a new user
- ▶ Username must be unique
- ▶ Password is encrypted before storing
- ▶ User details are saved in the database



# Signup - Pseudocode

- ▶ CLASS AuthController
- ▶     FUNCTION signup(request)
- ▶         userDetails = request data
- ▶         RETURN
- ▶         AuthService.registerUser(userDetails)
- ▶     END FUNCTION
- ▶ END CLASS

- ▶ CLASS AuthService
- ▶     FUNCTION registerUser(userDetails)
- ▶         IF username exists THEN
- ▶             RETURN "User already exists"
- ▶         END IF
- ▶         encryptedPassword =
- ▶         encrypt(password)
- ▶         create new User
- ▶         save User
- ▶         RETURN "Signup successful"
- ▶     END FUNCTION
- ▶ END CLASS

# Signin - Purpose

- ▶ **Explanation:**
- ▶ User logs in using username and password
- ▶ System verifies credentials
- ▶ JWT token is generated
- ▶ Token is used for authorization in future requests

# Signin - Pseudocode

- ▶ CLASS AuthController
- ▶   FUNCTION signin(request)
- ▶     RETURN AuthService.authenticate(  
▶       request.username,  
▶       request.password  
▶    )
- ▶   END FUNCTION
- ▶ END CLASS

- ▶ CLASS AuthService
- ▶   FUNCTION authenticate(username,  
▶     password)
- ▶     user = find user by username
- ▶     IF password matches THEN
- ▶       RETURN  
▶       JWTUtility.generateToken(user)
- ▶     ELSE
- ▶       RETURN "Invalid credentials"
- ▶     END IF
- ▶   END FUNCTION
- ▶ END CLASS

# Forgot Password - Purpose

- ▶ **Explanation:**
- ▶ Helps users reset password securely
- ▶ Avoids password exposure
- ▶ Reset is done using email verification





# Forgot Password - Pseudocode

- ▶ CLASS AuthController
  - ▶ FUNCTION forgotPassword(request)
    - ▶ RETURN AuthService.processForgotPassword(request.email)
  - ▶ END FUNCTION
- ▶ END CLASS
- ▶ CLASS AuthService
  - ▶ FUNCTION processForgotPassword(email)
    - ▶ generate reset token
    - ▶ send email with reset link
    - ▶ RETURN "Reset link sent"
  - ▶ END FUNCTION
- ▶ END CLASS

# Reset Password - Pseudocode

- ▶ CLASS AuthService
- ▶   FUNCTION resetPassword(token, newPassword)
- ▶     IF token invalid THEN
- ▶       RETURN "Invalid token"
- ▶     END IF
- ▶     encrypt new password
- ▶     save updated password
- ▶     RETURN "Password reset successful"
- ▶   END FUNCTION
- ▶ END CLASS

# Authentication Flow

## Explanation:

- Authentication follows a layered flow
- Each layer has a clear responsibility
- Business logic is separated from UI and database

## Flow:

- **Signup:**  
UI → AuthController → AuthService → UserRepository → Database
- **Signin:**  
UI → AuthController → AuthService → JWT Token → UI
- **Forgot Password:**  
UI → AuthController → AuthService → EmailService → User

# MODULE 2: PRODUCT / INVENTORY MANAGEMENT- Overview

- ▶ **Explanation:**
- ▶ Manages products in the system
- ▶ Tracks stock levels
- ▶ Prevents duplicate products
- ▶ Ensures inventory accuracy



# Product Entity

- ▶ **Explanation:**
- ▶ Represents product details
- ▶ SKU ensures uniqueness
- ▶ Stock quantity changes frequently
- ▶ Minimum stock level triggers alerts



# Product Entity Pseudocode:

- ▶ CLASS Product
- ▶     productId
- ▶     sku
- ▶     name
- ▶     unitPrice
- ▶     stockQuantity
- ▶     minStockLevel
- ▶ END CLASS

# Product Controller

- ▶ **Explanation:**
- ▶ Receives requests from UI
- ▶ Forwards requests to service layer
- ▶ Does not access database directly



# Product Controller Pseudocode:

- ▶ CLASS ProductController
- ▶   FUNCTION addProduct(productData)
- ▶     ProductService.addProduct(productData)
- ▶   END FUNCTION
  
- ▶   FUNCTION stockIn(id, qty)
- ▶     ProductService.increaseStock(id, qty)
- ▶   END FUNCTION
  
- ▶   FUNCTION stockOut(id, qty)
- ▶     ProductService.decreaseStock(id, qty)
- ▶   END FUNCTION
- ▶ END CLASS



# ProductService

- ▶ **Explanation:**
- ▶ Central business logic
- ▶ Handles stock updates
- ▶ Logs every transaction
- ▶ Triggers low stock alerts



# ProductService Pseudocode

- ▶ CLASS ProductService
- ▶   FUNCTION addProduct(data)
- ▶     IF SKU exists THEN
- ▶       RETURN "Duplicate Product"
- ▶     END IF
- ▶     save product
- ▶   END FUNCTION
- ▶   FUNCTION increaseStock(id, qty)
- ▶     update stock
- ▶     log transaction
- ▶     check alerts
- ▶   END FUNCTION
- ▶   FUNCTION decreaseStock(id, qty)
- ▶     IF sufficient stock THEN
- ▶       update stock
- ▶     ELSE
- ▶       RETURN "Insufficient stock"
- ▶     END IF
- ▶   END FUNCTION
- ▶ END CLASS

# ProductRepository

## Explanation:

- Handles database operations
- Saves and retrieves product data

## Pseudocode:

- ▶ CLASS ProductRepository
- ▶     FUNCTION findById(id)
- ▶         RETURN product
- ▶     END FUNCTION
- ▶     FUNCTION save(product)
- ▶         store in database
- ▶     END FUNCTION
- ▶ END CLASS

## Module 1 Key Points

- Verifies user identity before access
- Allows only registered users to log in
- Encrypts passwords for security
- Generates JWT token after login
- Blocks unauthorized users early

## Module 2 Key Points

- Prevents duplicate products
- Maintains accurate stock
- Logs inventory transactions
- Supports alerts and reports



# Conclusion

- Module 1 ensures **secure user authentication**
- Module 2 manages **products and inventory operations**
- Role-based access prevents unauthorized actions
- Stock updates are handled safely with proper validation
- Together, both modules make the system **secure, reliable, and efficient**

THANK YOU

